

Fundamentos de Sistemas Neuro-Fuzzy

Da Lógica Fuzzy ao ANFIS com Aprendizado Híbrido

Notas de estudo — Teoria e Aplicações de Inteligência Computacional
PPGEE/UFMS

Resumo

Estas notas constroem, do zero e com profundidade, todo o arcabouço teórico necessário para compreender e implementar um *Adaptive Neuro-Fuzzy Inference System* (ANFIS). Partimos da lógica fuzzy (conjuntos, funções de pertinência, t-normas), passamos pelos sistemas de inferência de Takagi–Sugeno, revisamos os fundamentos de otimização por gradiente e diferenciação automática, e culminamos na arquitetura de cinco camadas do ANFIS e no seu algoritmo de *aprendizado híbrido*, que combina mínimos quadrados (para os parâmetros consequentes) com *backpropagation* (para os parâmetros das funções de pertinência). As derivações que sustentam o treinamento — as derivadas parciais da função gaussiana, a regra da cadeia da rede, as equações normais dos mínimos quadrados e a reparametrização $\sigma = e^\lambda$ — são desenvolvidas em detalhe.

Sumário

1	Introdução e visão geral	1
1.1	O problema	1
1.2	Mapa do documento	1
2	Lógica fuzzy	2
2.1	Conjuntos clássicos e a fronteira rígida	2
2.2	Conjuntos fuzzy	2
2.3	Funções de pertinência gaussianas	3
2.4	Variáveis linguísticas e granularização	4
2.5	Operadores fuzzy: t-normas	4
3	Sistemas de inferência fuzzy	5
3.1	Estrutura geral	5
3.2	Mamdani <i>versus</i> Sugeno	5
3.3	Sugeno de ordem zero e de primeira ordem	5
3.4	As quatro regras do problema	5
3.5	O pipeline de inferência	6
4	Fundamentos de redes neurais e otimização	6
4.1	Função de custo	6
4.2	Gradiente descendente	6
4.3	Regra da cadeia e <i>backpropagation</i>	7
4.4	Diferenciação automática (<i>autograd</i>)	7
5	A arquitetura ANFIS	7
5.1	Visão em cinco camadas	7
5.2	Descrição camada a camada	8

6	Aprendizado híbrido	8
6.1	A intuição: dividir para conquistar	8
6.2	Linearidade nos consequentes	9
6.3	Mínimos quadrados	9
6.4	A passada para trás: gradiente nas premissas	9
6.5	Reparametrização $\sigma = e^\lambda$	10
6.6	O ciclo completo	10
6.7	Por que recalcular os consequentes a cada época?	11
7	Avaliação e generalização	11
7.1	Métricas	11
7.2	Generalização <i>versus</i> sobreajuste	11
A	Nomenclatura	12

1 Introdução e visão geral

1.1 O problema

Queremos estimar a velocidade desejada V (em %) de um ventilador a partir de duas grandezas ambientais: a temperatura T (em °C) e a umidade relativa U (em %). Dispomos de um conjunto de dados com pares de entrada-saída medidos. Trata-se, portanto, de um problema de *regressão*: aprender uma função $\hat{V} = F(T, U)$ que reproduza bem os dados observados e generalize para casos novos.

O que distingue a abordagem deste texto é a *forma* escolhida para F . Em vez de uma rede neural genérica (uma “caixa-preta”), usamos um sistema de inferência fuzzy, cuja estrutura é interpretável — ela codifica regras do tipo “se a temperatura é alta e a umidade é baixa, então a velocidade é tal” — mas cujos parâmetros são *aprendidos* dos dados, como numa rede neural. Essa fusão é o ANFIS.

Ideia-chave

ANFIS é um sistema fuzzy de Takagi–Sugeno cujos parâmetros são ajustados por técnicas de redes neurais. Ele herda do fuzzy a **interpretabilidade** (regras linguísticas) e do neuro a **capacidade de aprendizado** a partir de dados.

1.2 Mapa do documento

As Seções 2 e 3 desenvolvem o lado *fuzzy*: conjuntos, funções de pertinência, operadores e o mecanismo de inferência de Sugeno. A Seção 4 revisa o lado *neuro*: custo, gradiente, regra da cadeia e diferenciação automática. A Seção 5 monta a arquitetura ANFIS, mostrando que ela é exatamente o sistema Sugeno redesenhado como uma rede em camadas. A Seção 6 apresenta o coração do método — o aprendizado híbrido — com todas as derivações. A Seção 7 trata de avaliação e generalização.

2 Lógica fuzzy

2.1 Conjuntos clássicos e a fronteira rígida

Na teoria clássica de conjuntos, a pertinência de um elemento x a um conjunto A é uma questão binária, descrita pela *função característica*

$$\chi_A(x) = \begin{cases} 1, & x \in A, \\ 0, & x \notin A. \end{cases}$$

Se definirmos “temperatura alta” como “ $T > 30^\circ\text{C}$ ”, então $30,1^\circ\text{C}$ é totalmente alta e $29,9^\circ\text{C}$ não é nada alta. Essa descontinuidade abrupta não corresponde à forma como raciocinamos sobre grandezas contínuas: na prática, 29°C é “quase alta”, e gostaríamos de representar essa transição suave.

2.2 Conjuntos fuzzy

A lógica fuzzy, proposta por Zadeh (1965), substitui a pertinência binária por um *grau* de pertinência contínuo.

Definição — Conjunto fuzzy

Seja X um *universo de discurso* (o conjunto de todos os valores possíveis da variável, p. ex. todas as temperaturas). Um conjunto fuzzy A sobre X é caracterizado por uma *função de pertinência*

$$\mu_A : X \rightarrow [0, 1],$$

que associa a cada $x \in X$ um grau de pertinência $\mu_A(x) \in [0, 1]$. Formalmente, $A = \{(x, \mu_A(x)) \mid x \in X\}$.

O conjunto clássico é o caso particular em que μ_A só assume os valores 0 ou 1. O valor $\mu_A(x) = 0,6$ significa que x pertence a A em grau 0,6.



Figura 1: Pertinência ao conceito “temperatura alta”. À esquerda, a função característica clássica; à direita, uma função de pertinência fuzzy.

Atenção

Grau de pertinência **não** é probabilidade. $\mu_A(29) = 0,6$ não afirma “60% de chance de ser alta”; afirma que 29°C é alta em grau 0,6. A incerteza modelada é de *vagueza* (o conceito “alta” não tem fronteira nítida), não de *aleatoriedade*. Consequentemente, os graus das diferentes categorias não precisam somar 1.

2.3 Funções de pertinência gaussianas

Como o universo X é contínuo (infinitos valores), não armazenamos uma tabela de pertinências: usamos uma função *parametrizada*. Neste trabalho, adotamos a gaussiana

$$\mu(x) = \exp\left(-\frac{1}{2}\left(\frac{x-c}{\sigma}\right)^2\right), \quad \sigma > 0. \quad (1)$$

Ela possui dois parâmetros com significado geométrico claro:

- c — o **centro**: ponto de pertinência máxima, $\mu(c) = 1$. Representa o “valor típico” do conceito.
- σ — a **largura**: controla a dispersão. Valores grandes de σ produzem curvas largas e tolerantes; valores pequenos, curvas estreitas e exigentes.

Nos pontos $x = c \pm \sigma$ a pertinência vale sempre

$$\mu(c \pm \sigma) = \exp\left(-\frac{1}{2} \cdot 1^2\right) = e^{-1/2} \approx 0,6065,$$

independentemente de c e σ — um marco geométrico útil para leitura de gráficos.

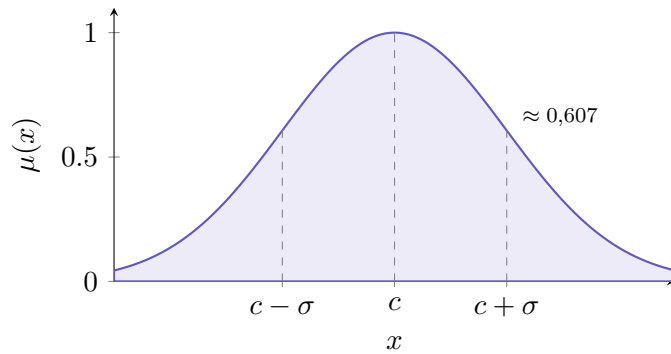


Figura 2: Função de pertinência gaussiana com $c = 50$ e $\sigma = 20$.

As derivadas que o treinamento vai usar. Como veremos, o *backpropagation* ajusta c e σ e, para isso, precisa das derivadas parciais de (1). Vale calculá-las agora. Escrevendo $z = (x-c)/\sigma$, temos $\mu = e^{-z^2/2}$ e $\frac{d\mu}{dz} = -z e^{-z^2/2} = -z \mu$. Pela regra da cadeia,

$$\frac{\partial \mu}{\partial c} = -z \mu \cdot \frac{\partial z}{\partial c} = -z \mu \cdot \left(-\frac{1}{\sigma}\right) = \mu(x) \frac{x-c}{\sigma^2}, \quad (2)$$

$$\frac{\partial \mu}{\partial \sigma} = -z \mu \cdot \frac{\partial z}{\partial \sigma} = -z \mu \cdot \left(-\frac{x-c}{\sigma^2}\right) = \mu(x) \frac{(x-c)^2}{\sigma^3}. \quad (3)$$

Repare no significado: (2) é positiva quando $x > c$ (aumentar o centro aproxima a curva de x e eleva a pertinência) e (3) é sempre não-negativa (alargar a curva nunca reduz a pertinência de um ponto fora do centro). Guardaremos estas expressões para a Seção 6.4.

2.4 Variáveis linguísticas e granularização

Uma *variável linguística* é uma variável cujos “valores” são termos linguísticos — p.ex. a temperatura assume os valores “baixa” e “alta”. Cada termo é um conjunto fuzzy com sua própria função de pertinência. Neste problema usamos **duas** funções de pertinência por entrada:

$$\text{Temperatura} \rightarrow A_1 \text{ (baixa)}, A_2 \text{ (alta)}; \quad \text{Umidade} \rightarrow B_1 \text{ (baixa)}, B_2 \text{ (alta)}.$$

São, portanto, quatro gaussianas, com oito parâmetros de premissa ao todo (c e σ de cada uma). Os valores iniciais adotados são os da Tabela 1.

Função de pertinência	c	σ
A_1 — Temperatura baixa	20	5
A_2 — Temperatura alta	35	5
B_1 — Umidade baixa	30	10
B_2 — Umidade alta	70	10

Tabela 1: Parâmetros iniciais das funções de pertinência.

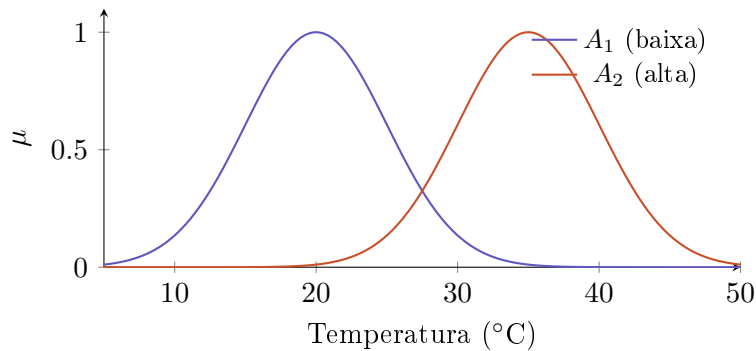


Figura 3: As duas funções de pertinência da temperatura (valores iniciais). Observe a sobreposição: temperaturas intermediárias pertencem, em graus distintos, a ambos os conceitos.

2.5 Operadores fuzzy: t-normas

Para combinar condições — “ T é A_1 E U é B_1 ” — precisamos de uma operação de conjunção fuzzy, chamada *t-norma*.

Definição — t-norma

Uma t-norma é uma função $\top : [0, 1]^2 \rightarrow [0, 1]$ comutativa, associativa, monótona (não-decrescente em cada argumento) e com elemento neutro 1, isto é, $\top(a, 1) = a$.

As t-normas mais comuns são o *mínimo*, $\top(a, b) = \min(a, b)$, e o *produto*, $\top(a, b) = a \cdot b$. Este trabalho usa o **produto**. A escolha não é arbitrária:

Ideia-chave

O produto é **suave e diferenciável** em todo o domínio, ao passo que o mínimo apresenta “quinas” (não é diferenciável onde $a = b$). Como o ANFIS ajusta seus parâmetros por gradiente, uma t-norma diferenciável fornece gradientes bem-definidos em toda parte — propriedade decisiva para o treinamento.

Assim, a *força de disparo* de uma regra — o quanto ela está ativa para uma dada entrada — é o produto dos graus de pertinência de suas condições. Em símbolos, para a Regra 1: $w_1 = \mu_{A_1}(T) \mu_{B_1}(U)$.

3 Sistemas de inferência fuzzy

3.1 Estrutura geral

Um sistema de inferência fuzzy (FIS) transforma entradas numéricas em uma saída numérica por meio de quatro estágios: (i) *fuzzificação* — calcular os graus de pertinência das entradas; (ii) *avaliação das regras* — calcular a força de disparo de cada regra; (iii) *agregação* — combinar as saídas das regras; e, em alguns modelos, (iv) *defuzzificação* — converter um resultado fuzzy de volta em número.

3.2 Mamdani *versus* Sugeno

Há dois grandes paradigmas de FIS, que diferem na *forma do consequente* das regras.

Aspecto	Mamdani	Takagi–Sugeno
Consequente da regra	conjunto fuzzy	função numérica das entradas
Defuzzificação	necessária (centroide)	dispensada
Custo computacional	maior	menor
Adequação a otimização	menor	alta (base do ANFIS)

Tabela 2: Comparação entre os modelos de Mamdani e de Sugeno.

No modelo de Sugeno, a saída de cada regra já é um número (uma função das entradas), de modo que a saída global é uma *média ponderada* — sem a etapa custosa de defuzzificação. Essa simplicidade e a natureza analítica dos consequentes tornam o Sugeno o alicerce natural do ANFIS.

3.3 Sugeno de ordem zero e de primeira ordem

No Sugeno de *ordem zero*, o consequente é uma constante, $f_i = r_i$. No de *primeira ordem* — o adotado aqui — o consequente é uma função *afim* (linear com termo independente) das entradas:

$$f_i = p_i T + q_i U + r_i. \quad (4)$$

Os coeficientes p_i, q_i, r_i são os *parâmetros consequentes*. Com quatro regras, são $4 \times 3 = 12$ parâmetros consequentes. A linearidade de (4) nos parâmetros — e não nas entradas — é a propriedade que, na Seção 6.3, permitirá resolvê-los por mínimos quadrados.

3.4 As quatro regras do problema

Combinando os dois termos de cada entrada (produto cartesiano 2×2), obtemos quatro regras:

Regra	Condição	Consequente
1	SE T é A_1 E U é B_1	$f_1 = p_1 T + q_1 U + r_1$
2	SE T é A_1 E U é B_2	$f_2 = p_2 T + q_2 U + r_2$
3	SE T é A_2 E U é B_1	$f_3 = p_3 T + q_3 U + r_3$
4	SE T é A_2 E U é B_2	$f_4 = p_4 T + q_4 U + r_4$

3.5 O pipeline de inferência

Para uma amostra (T, U) , a saída estimada é computada em quatro etapas.

Etapas 1 — Fuzzificação. Calcular os quatro graus de pertinência $\mu_{A_1}(T)$, $\mu_{A_2}(T)$, $\mu_{B_1}(U)$, $\mu_{B_2}(U)$ via (1).

Etapa 2 — Força de disparo (produto).

$$w_1 = \mu_{A_1}\mu_{B_1}, \quad w_2 = \mu_{A_1}\mu_{B_2}, \quad w_3 = \mu_{A_2}\mu_{B_1}, \quad w_4 = \mu_{A_2}\mu_{B_2}. \quad (5)$$

Etapa 3 — Normalização.

$$\bar{w}_i = \frac{w_i}{\sum_{j=1}^4 w_j}, \quad i = 1, \dots, 4, \quad (6)$$

de modo que $\sum_i \bar{w}_i = 1$: cada regra contribui na proporção de sua ativação relativa.

Etapa 4 — Agregação (saída).

$$\hat{V} = \sum_{i=1}^4 \bar{w}_i f_i. \quad (7)$$

4 Fundamentos de redes neurais e otimização

4.1 Função de custo

Aprender significa ajustar parâmetros para que $\hat{V}$ se aproxime de V . Medimos o desacordo pelo *erro quadrático médio* sobre as N amostras:

$$E = \frac{1}{N} \sum_{k=1}^N (V_k - \hat{V}_k)^2. \quad (8)$$

O erro é quadrático (penaliza mais os desvios grandes) e diferenciável, o que o torna adequado à otimização por gradiente.

4.2 Gradiente descendente

Seja θ um parâmetro qualquer. O gradiente $\partial E / \partial \theta$ indica a direção de *crescimento* do erro; para reduzi-lo, damos um passo no sentido oposto:

$$\theta \leftarrow \theta - \eta \frac{\partial E}{\partial \theta}, \quad (9)$$

onde $\eta > 0$ é a *taxa de aprendizado* (o tamanho do passo). Repetindo (9) ao longo de muitas *épocas*, o erro tende a um mínimo (Figura 4). Taxas muito grandes provocam oscilação ou divergência; muito pequenas, convergência lenta.

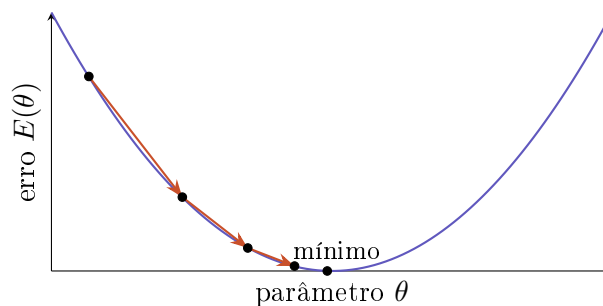


Figura 4: Gradiente descendente: passos sucessivos “descem” a superfície de erro em direção ao mínimo.

4.3 Regra da cadeia e *backpropagation*

O *backpropagation* não é senão a aplicação sistemática da regra da cadeia para obter $\partial E/\partial \theta$ de cada parâmetro, propagando a sensibilidade do erro da saída para as entradas. Para um centro de premissa, digamos c_{A_1} , a cadeia atravessa todas as camadas do pipeline da Seção 3.5. Seus “elos” são:

$$\begin{aligned} \frac{\partial E}{\partial \hat{V}_k} &= -\frac{2}{N} (V_k - \hat{V}_k), & \frac{\partial \hat{V}}{\partial \bar{w}_i} &= f_i, \\ \frac{\partial \bar{w}_i}{\partial w_j} &= \frac{\delta_{ij} - \bar{w}_i}{S}, \quad S = \sum_j w_j, & \frac{\partial w_1}{\partial \mu_{A_1}} &= \mu_{B_1}, \quad \frac{\partial w_2}{\partial \mu_{A_1}} = \mu_{B_2}, \end{aligned}$$

e, finalmente, $\partial \mu_{A_1}/\partial c_{A_1}$ dada por (2). (O termo $(\delta_{ij} - \bar{w}_i)/S$ vem de derivar o quociente (6); δ_{ij} é o delta de Kronecker.) Compondo os elos e somando sobre as regras que contêm A_1 (a 1 e a 2) e sobre as amostras, obtém-se $\partial E/\partial c_{A_1}$. O mesmo vale para σ , usando (3).

Ideia-chave

Na prática, **não** escrevemos essas derivadas à mão. Basta implementar a passada *para frente* (o pipeline); a biblioteca de *diferenciação automática* calcula todos os gradientes exatamente.

4.4 Diferenciação automática (*autograd*)

Ao executar o forward, o PyTorch registra cada operação num *grafo computacional* dirigido, cujas folhas são os parâmetros marcados como treináveis (`requires_grad=True`). Ao chamar `E.backward()`, ele percorre o grafo em sentido reverso (*reverse-mode AD*), aplicando a regra da cadeia elo a elo e acumulando o gradiente de E em relação a cada folha. É exato (não é diferença finita) e eficiente (um único percurso reverso calcula todos os gradientes).

5 A arquitetura ANFIS

5.1 Visão em cinco camadas

O sistema Sugeno da Seção 3 pode ser redesenhado como uma rede de cinco camadas (Jang, 1993), em que cada camada realiza exatamente uma etapa da inferência (Figura 5). A leitura da rede da esquerda para a direita reproduz o pipeline; a novidade é enxergá-la como uma estrutura *treinável*.

5.2 Descrição camada a camada

Camada 1 — Fuzzificação (adaptativa).

Cada nó calcula um grau de pertinência via (1). Os parâmetros são c e σ (*parâmetros de premissa*).

Camada 2 — Produto (fixa).

Cada nó multiplica pertinências para formar a força de disparo w_i . Sem parâmetros.

Camada 3 — Normalização (fixa).

Cada nó computa $\bar{w}_i$ por (6). Sem parâmetros.

Camada 4 — Consequente (adaptativa).

Cada nó calcula $\bar{w}_i f_i$, com f_i dado por (4). Os parâmetros são p_i, q_i, r_i (*parâmetros consequentes*).

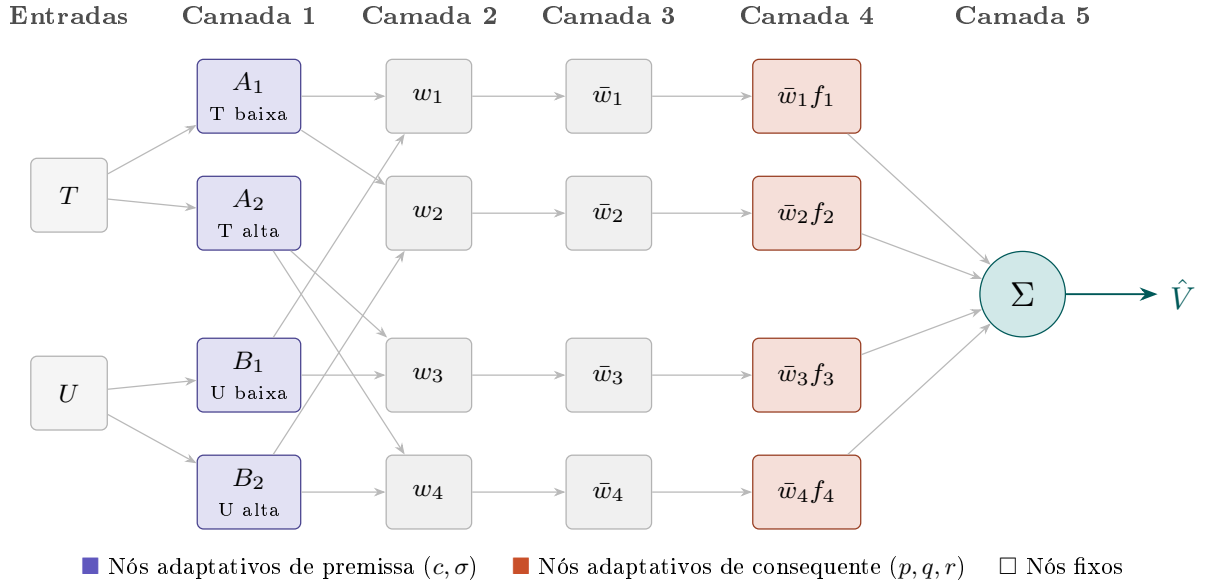


Figura 5: Arquitetura ANFIS de cinco camadas.

Camada 5 — Soma (fixa).

Um único nó soma as contribuições e entrega $\hat{V}$ por (7).

Ideia-chave

A rede tem duas famílias de parâmetros de **naturezas distintas**: os de premissa (camada 1) entram de forma **não-linear** em $\hat{V}$; os de consequente (camada 4) entram de forma **linear**. Essa dicotomia é o que viabiliza o aprendizado híbrido.

6 Aprendizado híbrido

6.1 A intuição: dividir para conquistar

Poderíamos ajustar todos os $8 + 12 = 20$ parâmetros por gradiente descendente. Funciona, mas converge lentamente, pois o problema conjunto é fortemente não-linear. O algoritmo híbrido de Jang explora a estrutura: fixados os parâmetros de premissa, o problema nos consequentes é *linear* e admite solução *exata* por mínimos quadrados. Divide-se então cada época em duas passadas:

- **Passada para frente:** com as premissas atuais, resolver os consequentes por mínimos quadrados (parte linear).
- **Passada para trás:** com os consequentes fixos, ajustar as premissas por *backpropagation* (parte não-linear).

6.2 Linearidade nos consequentes

Substituindo (4) em (7), para a amostra k :

$$\hat{V}_k = \sum_{i=1}^4 \bar{w}_{i,k} (p_i T_k + q_i U_k + r_i). \quad (10)$$

Distribuindo, cada termo é um *coeficiente conhecido* (dependente de $\bar{w}$, T , U) multiplicando um *parâmetro desconhecido*:

$$\hat{V}_k = (\bar{w}_{1,k} T_k) p_1 + (\bar{w}_{1,k} U_k) q_1 + (\bar{w}_{1,k}) r_1 + \cdots + (\bar{w}_{4,k}) r_4. \quad (11)$$

Ou seja, fixadas as premissas, $\hat{V}_k$ é **linear** nos doze consequentes.

6.3 Mínimos quadrados

Empilhando as N amostras de treino, a Equação (11) vira o sistema

$$\Phi \theta = V, \quad (12)$$

com $\theta = [p_1, q_1, r_1, \dots, p_4, q_4, r_4]^\top \in \mathbb{R}^{12}$, $V = [V_1, \dots, V_N]^\top \in \mathbb{R}^N$, e a *matriz de projeto* $\Phi \in \mathbb{R}^{N \times 12}$ cuja k -ésima linha é

$$\Phi_{k,:} = [\bar{w}_1 T, \bar{w}_1 U, \bar{w}_1, \bar{w}_2 T, \bar{w}_2 U, \bar{w}_2, \bar{w}_3 T, \bar{w}_3 U, \bar{w}_3, \bar{w}_4 T, \bar{w}_4 U, \bar{w}_4]_k. \quad (13)$$

Como $N = 800 > 12$, o sistema é *sobredeterminado*: em geral não há θ que satisfaça todas as equações. Buscamos então a solução de *mínimos quadrados*, que minimiza o resíduo

$$J(\theta) = \|\Phi \theta - V\|_2^2. \quad (14)$$

Anulando o gradiente, $\nabla_\theta J = 2\Phi^\top(\Phi \theta - V) = 0$, chegamos às *equações normais*:

$$\Phi^\top \Phi \theta = \Phi^\top V \implies \theta^* = (\Phi^\top \Phi)^{-1} \Phi^\top V = \Phi^+ V, \quad (15)$$

onde Φ^+ é a *pseudo-inversa* de Moore–Penrose.

Atenção

Não implemente (15) formando $(\Phi^\top \Phi)^{-1}$ explicitamente. O número de condição satisfaz $\kappa(\Phi^\top \Phi) = \kappa(\Phi)^2$: montar as equações normais *eleva ao quadrado* o mau-condicionamento, amplificando erros de arredondamento. Prefira um solucionador baseado em fatoração QR ou SVD — em PyTorch, `torch.linalg.lstsq(Phi, V)` — que resolve o problema de forma numericamente estável.

6.4 A passada para trás: gradiente nas premissas

Resolvidos os consequentes, calcula-se $\hat{V}$ por (7) e o erro E por (8). A chamada a `E.backward()` propaga o gradiente até os oito parâmetros de premissa (usando, internamente, os elos da regra da cadeia e as derivadas (2)–(3)), e o otimizador aplica (9). O efeito geométrico é intuitivo: os centros c deslizam para as regiões onde os dados se concentram, e as larguras σ se ajustam para cobrir a dispersão de cada faixa.

6.5 Reparametrização $\sigma = e^\lambda$

A largura σ é, por definição, positiva. Contudo, o gradiente descendente (9) é irrestrito e pode produzir um passo que torne $\sigma \leq 0$, o que invalida a gaussiana (largura nula ou negativa, divisão por zero). A solução é *reparametrizar*: em vez de tratar σ como parâmetro livre, adota-se um parâmetro auxiliar $\lambda \in \mathbb{R}$ e define-se

$$\sigma = e^\lambda, \quad \lambda \in \mathbb{R}. \quad (16)$$

Como a exponencial é positiva para todo real, garante-se $\sigma > 0$ *por construção*, qualquer que seja o valor (positivo ou negativo) que o gradiente atribua a λ . Por exemplo, $\lambda = \ln 5 \Rightarrow \sigma = 5$ e $\lambda = -2 \Rightarrow \sigma = e^{-2} \approx 0,135$.

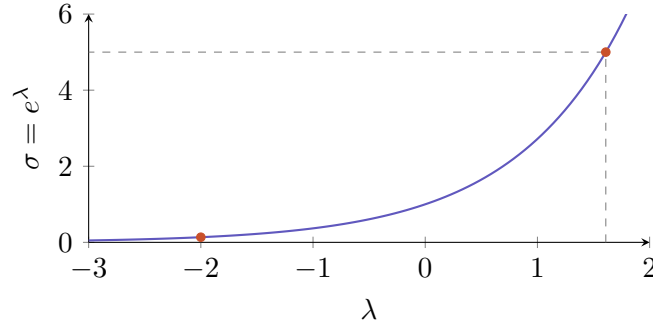


Figura 6: A reparametrização $\sigma = e^\lambda$ mantém $\sigma > 0$ para qualquer $\lambda \in \mathbb{R}$.

A reparametrização é transparente para o *autograd*: como $d\sigma/d\lambda = e^\lambda = \sigma$, o gradiente em relação a λ decorre de (3) pela regra da cadeia,

$$\frac{\partial \mu}{\partial \lambda} = \frac{\partial \mu}{\partial \sigma} \cdot \frac{d\sigma}{d\lambda} = \mu(x) \frac{(x-c)^2}{\sigma^3} \cdot \sigma = \mu(x) \frac{(x-c)^2}{\sigma^2}. \quad (17)$$

Na implementação, o parâmetro treinável é λ (inicializado como $\lambda_0 = \ln \sigma_0$) e a largura é recuperada por $\sigma = e^\lambda$ dentro do forward:

```
lam = torch.tensor(np.log(5.0), requires_grad=True) # sigma_inicial = 5
# ... dentro do forward:
sigma = torch.exp(lam) # sempre > 0
mu = torch.exp(-0.5 * ((x - c) / sigma)**2)
```

6.6 O ciclo completo

Cada época executa a sequência

Forward $\rightarrow$ Mínimos Quadrados $\rightarrow$ Saída $\rightarrow$ Erro $\rightarrow$ Backprop $\rightarrow$ Atualização,

resumida no Algoritmo a seguir.

Algoritmo — uma época de treinamento híbrido

1. **Forward:** com c e $\sigma = e^\lambda$ atuais, calcule as pertinências, as forças de disparo w_i e as normalizadas $\bar{w}_i$ (dados de treino).
2. **Mínimos quadrados:** monte Φ por (13) e resolva θ^* por (15) com um solucionador estável. Trate θ^* como *constante* na passada seguinte.
3. **Saída:** $\hat{V} = \Phi \theta^*$.
4. **Erro:** E por (8).
5. **Backprop:** `E.backward()` calcula $\partial E / \partial c$ e $\partial E / \partial \lambda$.
6. **Atualização:** o otimizador aplica (9) a c e λ .

6.7 Por que recalculer os consequentes a cada época?

A solução ótima dos consequentes depende das premissas: escrevendo a dependência explicitamente, $\theta^*(\pi) = \Phi(\pi)^+ V$, onde $\pi = \{c, \sigma\}$ denota o conjunto de parâmetros de premissa. Quando o passo de backprop altera π , a matriz $\Phi(\pi)$ muda (pois os $\bar{w}_i$ mudam) e, portanto, o θ^* anterior deixa de ser ótimo. Recalculer os consequentes a cada época mantém-nos sempre ótimos

para a configuração corrente das gaussianas. É essa realimentação — premissas movem os consequentes, que por sua vez definem o erro que move as premissas — que confere ao híbrido sua rápida convergência.

7 Avaliação e generalização

7.1 Métricas

Concluído o treinamento (usando apenas as amostras de treino), avalia-se o modelo nas amostras de *teste*, jamais vistas durante o ajuste. Reporta-se

$$\text{MSE} = \frac{1}{N} \sum_{k=1}^N (V_k - \hat{V}_k)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}. \quad (18)$$

O RMSE tem a vantagem de estar na *mesma unidade* da saída (% de velocidade), o que o torna diretamente interpretável: é a magnitude típica do erro de estimativa.

7.2 Generalização *versus* sobreajuste

Comparar o desempenho em treino e teste diagnostica a *capacidade de generalização*:

- Se o erro de teste é próximo do de treino, o modelo **generaliza** bem — aprendeu a relação subjacente, não apenas os exemplos.
- Se o erro de teste é muito maior que o de treino, há **sobreajuste** (*overfitting*): o modelo “decorou” o treino e falha em dados novos.

Ideia-chave

Um bom resultado de ANFIS combina *baixo* erro de teste com erros de treino e teste *próximos entre si*. O primeiro atesta capacidade; o segundo, generalização.

A Nomenclatura

Símbolo	Significado
T, U, V	temperatura, umidade e velocidade (real)
$\hat{V}$	velocidade estimada pelo modelo
A_1, A_2	conjuntos fuzzy da temperatura (baixa, alta)
B_1, B_2	conjuntos fuzzy da umidade (baixa, alta)
$\mu(\cdot)$	função de pertinência (grau em $[0, 1]$)
c, σ	centro e largura da gaussiana (parâmetros de premissa)
λ	parâmetro auxiliar com $\sigma = e^\lambda$
$w_i, \bar{w}_i$	força de disparo e força normalizada da regra i
f_i	consequente da regra i , $f_i = p_i T + q_i U + r_i$
p_i, q_i, r_i	parâmetros consequentes da regra i
Φ, θ, V	matriz de projeto, vetor de consequentes, vetor de saídas
E	erro quadrático médio (custo)
η	taxa de aprendizado
N	número de amostras

Forward → *Mínimos Quadrados* → *Saída* → *Erro* → *Backpropagation* → *Atualização*.